Discrete Markov Process

System is at any time *t* in one of N states q_t.
At regularly spaced discrete times system undergoes a change of state according to set of probabilities associated with the state.
For the Markov process of first order probability of changing state depends only on current state.

Changing state S_i (at time *t-1*) to state S_j (at time *t*) is given by:
$P(q_t = S_j \mid q_{t-1} = S_i)$.

Since system state does not depend on time probability of state transition from *i* to *j* is:

$a_{ij} = P(q_t = S_j \mid q_{t-1} = S_i), \quad 1 \le i, j \le N$, where $a_{ij} \ge 0$ and $\sum_{j=1}^{N} a_{ij} = 1$

Observable Markov model – model state corresponds to an observable (physical) event.

Hidden Markov Model

Observable events **are not the states** of the process.
We know at the most the probability of making given observation in every of the process states.
Since we cannot directly observe state changes of the process, such a model is called *Hidden Markov Model* (*HMM*).

A very good introduction to the problem is:
Appendix A of *Speech and Language Processing* by Daniel Jurafsky & James H. Marting:
https://web.stanford.edu/~jurafsky/slp3/A.pdf
(checked on May 2nd 2026)

HMM Parameters

1. The number of states *N* in the model.
2. The number of distinct observations *M* (observation alphabet size)
3. The state transition probability matrix *A*: $A = \{a_{ij}\}$
4. Probability of symbol observation in state *i*:
   $B = \{b_i(k)\}, b_i(k) = P(v_k \mid q_t = S_i), \quad 1 \le i \le N, 1 \le k \le M$
5. The initial state distribution
   $\pi_i = P(q_1 = S_i), \quad 1 \le i \le N$

In compact notation we use $\lambda = (A, B, \pi)$; parameter values *N* (number of states) and *M* (number of symbols in observation alphabet) are not used explicitly here.

The Basic Problems for HMM

1. Given the observation sequence $O = O_1 O_2 \ldots O_T$ and model $\lambda = (A, B, \pi)$ compute the probability of the sequence, given the model $P(O|\lambda)$.
   Forward-Backward procedure
2. Given the observation sequence O and model $\lambda = (A, B, \pi)$ compute state sequence $Q = q_1 q_2 \cdots q_T$ which "best" explains the observations.
   Viterbi algorithm
3. Adjust model parameters $\lambda = (A, B, \pi)$ to maximize $P(\{O\}|\lambda)$ for observations set $\{O\}$.
   Baum-Welsh procedure

Computing P(O|λ)

A trivial exhausting search has computational complexity of $O(2T*N^T)$.

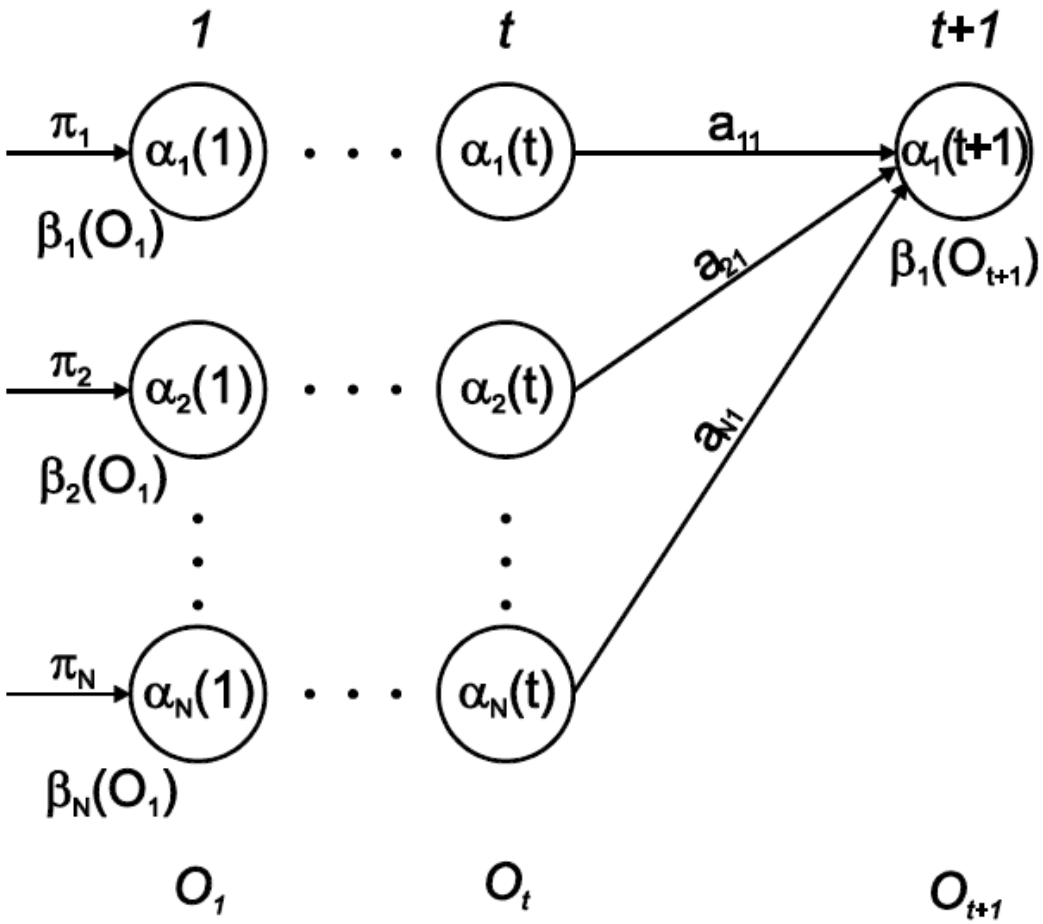
Forward-Backward procedure iteratively computes total probability of the observation sequence *forward*:

$$\alpha_t(i) = P(O_1 O_2 \ldots O_t, q_t = S_i | \lambda), 1 \le i \le N, 1 \le t \le T$$

and/or backward:

$$\beta_t(i) = P(O_{t+1} O_{t+2} \ldots O_T, q_t = S_i | \lambda), 1 \le i \le N, 1 \le t \le T$$

Illustration of single step of the procedure



Forward procedure

1. Initialization

$$\alpha_1(i) = \pi_i b_i(O_1), \quad 1 \le i \le N$$

2. Induction

$$\alpha_{t+1}(i) = \left[\sum_{j=1}^{N} \alpha_t(j) a_{ij} \right] b_i(O_{t+1}), \quad 1 \le i \le N, 1 \le t \le T-1$$

3. Termination

$$P(O|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$$

Backward procedure

1. Initialization

$$\beta_T(i) = 1, \quad 1 \le i \le N$$

2. Induction

$$\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(O_{t+1}) \beta_{t+1}(j), \quad 1 \le i \le N, 1 \le t \le T-1$$

3. Termination

$$P(O|\lambda) = \sum_{i=1}^{N} \pi_i b_i(O_1) \beta_1(i)$$

Viterbi Algorithm

Finding optimal state sequence $Q = q_1 q_2 \cdots q_T$ for given observation sequence $O = O_1 O_2 \ldots O_T$.

We define the variable $\delta_t(i) = \max_{q_1 q_2 \cdots q_{t-1}} P\left(q_1 q_2 \ldots q_t = S_i, O_1 O_2 \ldots O_t \middle| \lambda\right)$, which is the problem solution at time *t* and ends in state S_i. Trace variable (array) ψ will be used to backtrack state sequence, since δ describes only probability.

1. Initialization

   $\delta_1(i) = \pi_i b_i(O_1), 1 \le i \le N$

   $\psi_1(i) = 0, 1 \le i \le N$

2. Recursion

$$\delta_t(j) = \max_{1 \le i \le N} \left[\delta_{t-1}(i) a_{ij} \right] b_j(O_t), 1 \le i \le N, 2 \le t \le T$$

$$\psi_t(j) = \arg\max_{1 \le i \le N} \left[\delta_{t-1}(i) a_{ij} \right], 1 \le i \le N, 2 \le t \le T$$

3. Termination

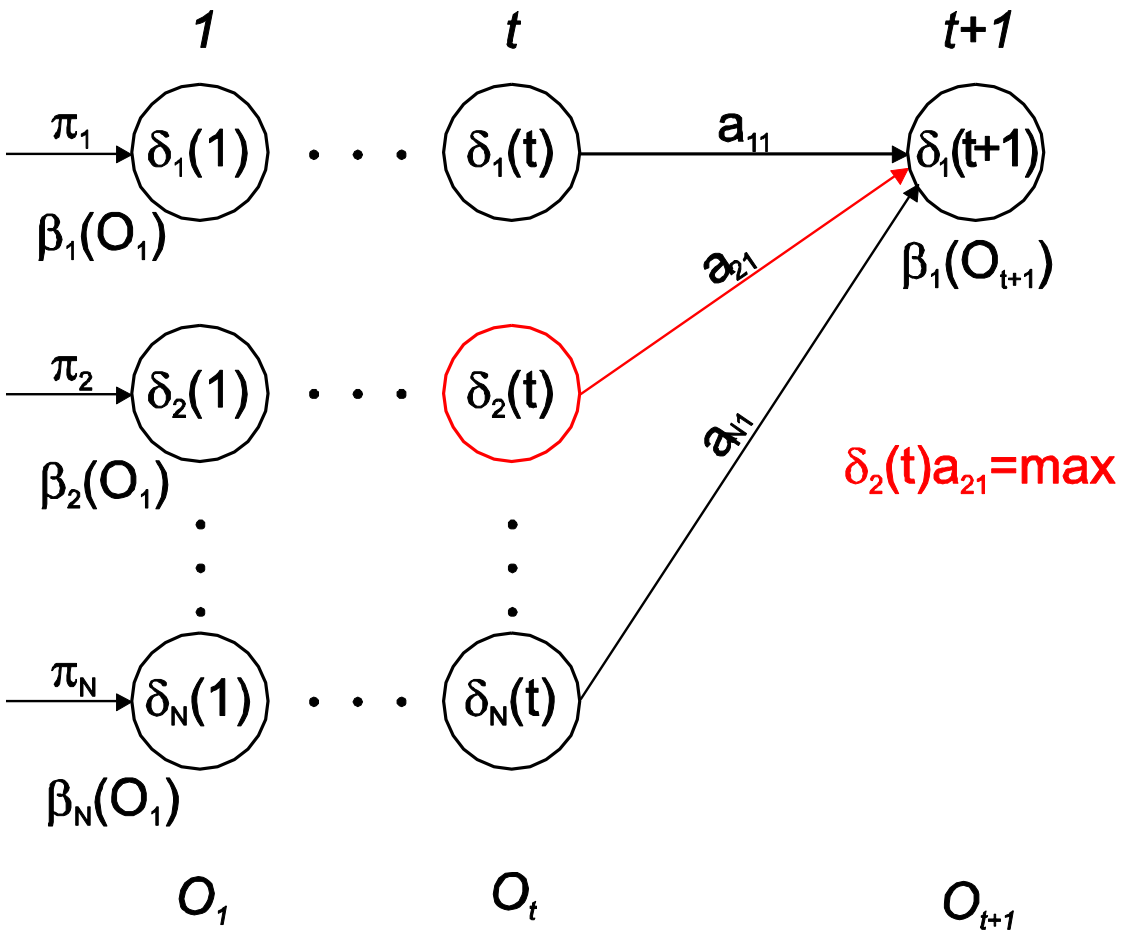
$$P^* = \max_{1 \le i \le N} \left[\delta_T(i) \right]$$

$$q_T^* = \arg\max_{1 \le i \le N} \left[\delta_T(i) \right]$$

4. State sequence backtracking

$$q_t^* = \psi_{t+1}\left(q_{t+1}^*\right), \quad t = T-1, T-2 \ldots 1$$

Illustration of single step of Viterbi algorithm



(Generally it is the same as in *forward* procedure, but here we are only interested in one “best” transition to the given state)

Baum-Welsh Algorithm

Computing model's parameters is not trivial problem. Difficulties start at the very beginning. It is relatively easy to assess the number of different observations, but the number of model's states is in general only the designer's presumption (excluding these rare cases when knowledge of the problem domain allows us to create reliable generative model).
Baum-Welsh algorithm relies on adjusting model's parameters to the observation sequence. We define auxiliary variable γ representing probability of transition from state *i* to *j* at time *t*:

$$\gamma_{ij}(t) = \frac{\alpha_i(t-1)a_{ij}b_{jk}\beta_j(t)}{P(O \mid \lambda)}, \quad 1 \le i\ j \le N, 1 \le k \le M, 2 \le t \le T-1$$

Baum-Welsh Algorithm

The value of variable γ is used to compute (better) estimates of model's parameters λ.
Estimate of the transition probability:

$$\hat{a}_{ij} = \frac{\sum_{t=1}^{T} \gamma_{ij}(t)}{\sum_{t=1}^{T} \sum_{k=1}^{N} \gamma_{ik}(t)}$$

Estimate of the probability of generating observable value:

$$\hat{b}_{ik} = \frac{\sum_{t=1}^{T} \sum_{l} \gamma_{ij}(t) \left| v(t) = v_k \right.}{\sum_{t=1}^{T} \sum_{l} \gamma_{il}(t)}$$

Recognition with strings

Let’s assume that objects are represented as ordered sequences of discrete elements.
Generally these elements (letters of some alphabet) are nominal data – there is no well-defined distance measure between letters.
The next difficulty results from different length of letters – words sequences.

Problems:

- String matching
- Edit distance
- String matching with errors
- String matching with ~ (don’t care) symbol

String matching

String matching consists in finding all appearances of the given *pattern* in *text*.
Pattern is denoted as ***x***=x[0..m-1]; length of pattern equals *m*.
Text is denoted as ***y***=y[0..n-1]; its length is equal *n*.
Pattern and text have letters from alphabet A, of size *a*.
String matching most often uses window (usually of size *m*). For given window position (with specific *shift* with respect to the beginning of the text) algorithm performs character matching *attempt*. (An attempt results in finding pattern in text or not). After an attempt window is moved toward the end of the text.

Differences between string matching algorithms lie in the method for determining shift (increment) after an attempt. A good starting point for further reading is: http://www-igm.univ-mlv.fr/~lecroq/string/
(checked on May 2nd 2026)

Practical approach

```
p = y - 1;
while ((p = strstr(p+1, x)) != 0)
   printf("%d\n", p-y);
```

This solution is optimal with respect to implementation time. It can be also quite fast, because `strstr` function is implemented neatly in the standard library.
(It would be instructive, to check implementation of `strstr` (or its counterpart in your favorite programming language)).

Naïve solution

```
for (int j=0; j <= n-m; ++j)
{
   int i = 0;
   for ( ; i < m && x[i] == y[i+j]; ++i);
   if (i == m)
      printf("%d\n", j);
}
```

No preprocessing needed.
Comparing from left to right.
Constant shift of 1.
Computational complexity $\boldsymbol{O}(mn)$.
Expected number of comparisons $2n$.

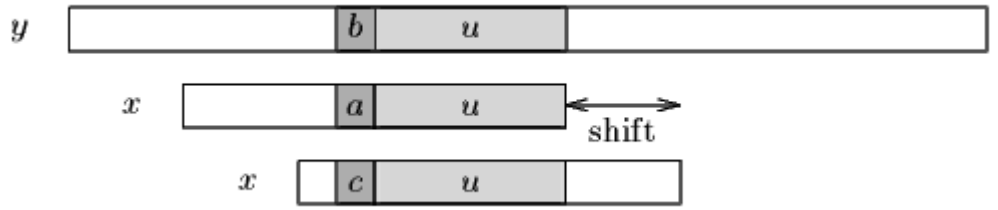
Boyer-Moore algorithm

Algorithm has preprocessing phase, during which shift arrays are calculated: good suffix array (Gs) and bad character array (Bc).

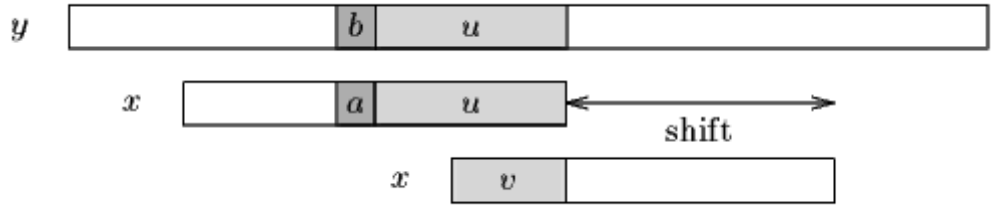
```
for (int j=0; j <= n-m;)
{
   int i = m-1;
   for ( ; i>=0 && x[i] == y[i+j]; --i);
   if (i < 0)
   {
      printf("%d\n", j);
      j += Gs[0];
   }
   else
      j += max(Gs[i], Bc[y[i+j]]-m+1+i);
}
```

BM algorithm – good suffix

We have found different characters x[i]=*a* and y[i+j]=*b*. The suffix x[i+1..m-1]=*u* is the same in pattern and text. The shift is computed as the rightmost occurrence of *u* preceded by other than *a* character.

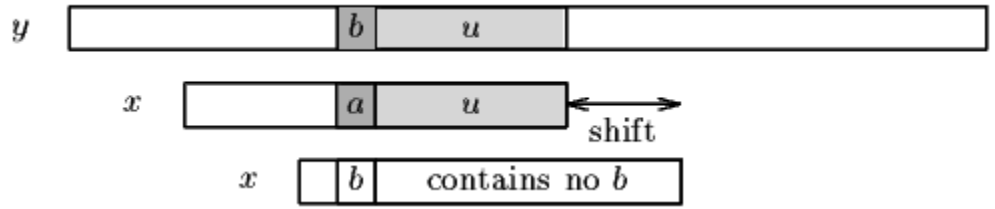


If *u* occurs in *x* only at the end, the shift results from the longest match between suffix *u* as the prefix of pattern *x*.

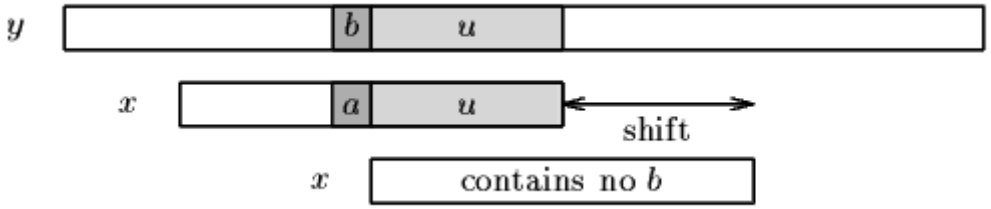


BM algorithm – bad character

Bad character shift aligns character *b*=y[i+j] with the rightmost occurrence of *b* in *x*[0..m-2].



If *b* does not occur in *x* at all, we shift the beginning of *x* after the occurrence of *b* in *y*, i.e. to the position i+j+1.



Boyer-Moore algorithm

BM algorithm is considered the most effective matching algorithm in ordinary applications.
Comparisons performed from right to left.
Preprocessing is $\boldsymbol{O}(m+a)$ in time and memory.
Computational complexity of matching $\boldsymbol{O}(mn)$.
Expected number of comparisons in the worst case: $3n$ (for aperiodic pattern).
Best case complexity $\boldsymbol{O}(n/m)$.

Example
Alphabet: a c g t
Pattern: `gcagagag`
Text: `gcatcgcagagagtatacagtacg`

Boyer-Moore - example

```
Bc:   a c g t                        Gs:   0 1 2 3 4 5 6 7
      1 6 2 8                              7 7 7 2 7 4 7 1

1. gcatcgcagagagtatacagtacg
   .......g                    s=1 (Gs[7]=Bc[a]-8+8)

2. gcatcgcagagagtatacagtacg
   . ....gAG                   s=4 (Gs[5]=Bc[c]-8+6)

3. gcatcgcagagagtatacagtacg
        GCAGAGAG               s=7 (Gs[0])

4. gcatcGCAGAGAGtatacagtacg
               .....gAG        s=4 (Gs[5]=Bc[c]-8+6)

5. gcatcGCAGAGAGtatacagtacg
                     ......aG  s=7 (Gs[6])
```

5 attempts, 17 character comparisons.

Edit distance (Levenshtein distance)

The distance between two strings is simply the number of elementary edit operations transforming string *x* into *y*. These elementary operations are:

- substitution
- deletion
- insertion

```
// initialization C: C[i,0]=i; C[0,j]=j
for (int i=0; i<m; i++)
   for (int j=0; j<n; j++)
      C[i+1, j+1] = min(C[i, j+1] + 1,// insertion
                        C[i+1, j] + 1, // deletion
                        C[i,j] + 1 - x[i] == y[j]);
                                       // subst.?
// C[m, n] is an edit distance
```

Edit distance

